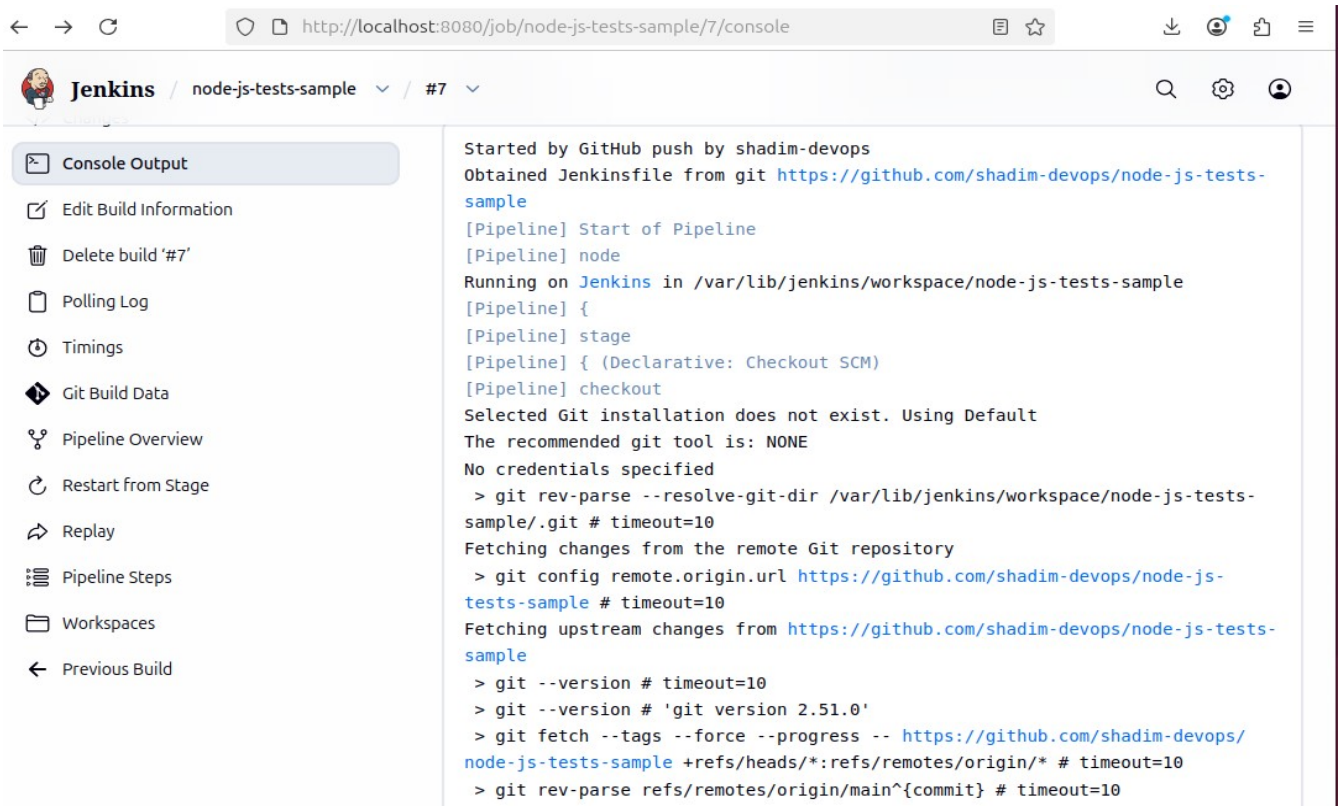
Zadanie 1: Zaawansowany pipeline z obsługą błędów i powiadomieniami

1. Console Output - Checkout stage



2. Console Output - Build stage

```
Checking out Revision 3b954c4615f1b4dabe0e220144b7b600d703ca89 (refs/remotes/
origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 3b954c4615f1b4dabe0e220144b7b600d703ca89 # timeout=10
Commit message: "Refactor email notifications in Jenkins pipeline"
 > git rev-list --no-walk 447b7bc8ec6dcb4c6ebf75d7b82cec4902a68d08 #
timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Downloading code...
[Pipeline] git
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
 > git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/node-js-tests-
sample/.git # timeout=10
Fetching changes from the remote Git repository
```

3.Console Output - Post actions

ns / node-js-tests-sample / #7

```
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Pipeline finished in 7 sec and counting
[Pipeline] }
[Pipeline] // script
[Pipeline] echo
Pipeline succeeded
[Pipeline] script
[Pipeline] {
[Pipeline] mail
[Pipeline] echo
Email failed
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

4.Full Stage View in Jenkins

http://localhost:8080/job/node-js-tests-sample/workflow-stage/

Jenkins / node-js-tests-sample / Full Stage View

node-js-tests-sample - Stage View

	Declarative: Checkout SCM	Checkout	Build	Test	Deploy	Declarative: Post Actions
Average stage times: (full run time: ~9s)	1s	1s	326ms	295ms	584ms	366ms
#7 Mar 29 23:09 1 commit	1s	1s	674ms	637ms	777ms	621ms

5.Jenkinsfile - Part 1 (environment + checkout)

```
Jenkinsfile ✕   ⓘ README.md ●
Jenkinsfile
pipeline {
    agent any

    environment {
        VERSION = "1.0"
        ENVIRONMENT = "dev"
        AUTHOR = "Dmytro"
        EMAIL = "email@example.com"
    }

    stages {

        stage('Checkout') {
            steps {
                script {
                    try {
                        echo "Downloading code..."
                        git url: 'https://github.com/shadim-devops/node-js-tests-sample.gi
                    } catch (e) {
                        echo "Checkout failed: ${e}"
                        currentBuild.result = 'FAILURE'
                    }
                }
            }
        }

        stage('Build') {
            steps {
                script {
                    try {
                        echo "Building version ${env.VERSION}"
                        sh 'echo "Build done"'
                    } catch (e) {
                        echo "Build failed: ${e}"
                        currentBuild.result = 'FAILURE'
                    }
```

6.Jenkinsfile - Part 2 (build + test)

```
        }

        stage('Test') {
            steps {
                script {
                    try {
                        echo "Running tests..."
                        sh 'echo "Tests passed"'
                    } catch (e) {
                        echo "Tests failed: ${e}"
                        currentBuild.result = 'UNSTABLE'
                    }
                }
            }
        }

        stage('Deploy') {
            when {
                expression {
                    return currentBuild.result == null || currentBuild.result == 'SUCCESS'
                }
            }
            steps {
                script {
                    try {
                        echo "Deploying to ${env.ENVIRONMENT}"
                        sh 'echo "Deploy done"'
                    } catch (e) {
                        echo "Deploy failed: ${e}"
                        currentBuild.result = 'FAILURE'
```

7.Jenkinsfile - Part 3 (deploy + post)

```
    }

    post {
        always {
            script {
                def duration = currentBuild.durationString
                echo "Pipeline finished in ${duration}"
            }
        }

        success {
            echo "Pipeline succeeded"
            mail to: "${env.EMAIL}",
                 subject: "SUCCESS: Pipeline completed",
                 body: """Pipeline SUCCESS

Author: ${env.AUTHOR}
Version: ${env.VERSION}
Environment: ${env.ENVIRONMENT}
Time: ${currentBuild.durationString}
Result: SUCCESS
"""
        }

        failure {
            echo "Pipeline failed"
            mail to: "${env.EMAIL}",
                 subject: "FAILED: Pipeline error",
                 body: """Pipeline FAILED

Author: ${env.AUTHOR}
Version: ${env.VERSION}
Environment: ${env.ENVIRONMENT}
Time: ${currentBuild.durationString}
Result: FAILURE
```

Raport końcowy

Został utworzony zaawansowany pipeline Jenkins składający się z czterech etapów: Checkout, Build, Test oraz Deploy. Pipeline integruje się z repozytorium GitHub.
W każdym etapie zaimplementowano obsługę błędów przy użyciu bloków try-catch, co pozwala na kontrolowanie przebiegu pipeline'u oraz ustawianie odpowiedniego statusu (SUCCESS, FAILURE, UNSTABLE).
Zastosowano warunkowe wykonanie etapu Deploy, który uruchamia się tylko w przypadku pomyślnego zakończenia poprzednich etapów.
Dodano sekcję post dla całego pipeline'u, w której realizowane są powiadomienia e-mail w przypadku sukcesu oraz błędu. Powiadomienia zawierają informacje o autorze, wersji, środowisku, czasie wykonania oraz wyniku pipelinu.
W pipeline wykorzystano zmienne środowiskowe do konfiguracji parametrów takich jak wersja, środowisko oraz adres e-mail.